
SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

EXPERIMENTS

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Calculator using YACC

Design a YACC grammar for arithmetic operators + and *. Use semantic actions to evaluate the expression $3 + 4 * 5$ and display the result.

Q1.1

```
%{  
  
#include "Q1.tab.h"  
  
%}  
  
%%  
  
[0-9]+ { yylval=atoi(yytext); return NUMBER; }  
  
[ \t] ;  
  
\n    return '\n';  
  
.    return yytext[0];  
  
%%
```

Q1.y

```
%{  
  
#include<stdio.h>  
  
int yylex();  
  
void yyerror(char *s);  
  
%}  
  
%token NUMBER  
  
%left '+'  
  
%left '*'  
  
%%  
  
input: expr '\n' { printf("Result = %d\n",$1); };  
  
expr:  
    expr '+' expr { $$=$1+$3; }
```

```

| expr '*' expr { $$=$1*$3; }

| NUMBER      { $$=$1; }

;

%%

void yyerror(char *s)

{

    printf("Syntax Error\n");

}

int main()

{

    printf("Enter expression: ");

    yyparse();

    return 0;

}

```

OUTPUT

```

C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kavet\Downloads\C03-AT1>Q1.exe
Enter expression: 2+5*8
Computed Result = 42
Q1.exe
Syntax Error: syntax error

C:\Users\kavet\Downloads\C03-AT1>Q1.exe
Enter expression: 3*6+5
Computed Result = 23
|

```

Q2. Abstract Syntax Tree using YACC

Define a YACC grammar for arithmetic expressions. Construct an Abstract Syntax Tree (AST) for $a + b * c$ using semantic actions and display its traversal order.

Q2.1

```
%{  
  
#include "Q2.tab.h"  
  
%}  
  
%%  
  
[a-zA-Z] { yylval=yytext[0]; return ID; }  
  
[ \t]    ;  
  
\n      return '\n';  
  
.       return yytext[0];  
  
%%
```

Q2.y

```
%{  
  
#include<stdio.h>  
  
#include<stdlib.h>  
  
typedef struct node  
{  
  
    char c;  
  
    struct node *l,*r;  
  
} Node;  
  
Node* make(char c,Node*l,Node*r)  
{  
  
    Node*n=malloc(sizeof(Node));  
  
    n->c=c; n->l=l; n->r=r;  
  
    return n;  
  
}
```

```

}

void preorder(Node*n)

{
    if(n){ printf("%c ",n->c); preorder(n->l); preorder(n->r); }
}

void yyerror(char*s){ printf("Syntax Error\n"); }

int yylex();

%}

%union { char c; Node *n; }

%token <c> ID

%type <n> expr

%left '+'

%left '*'

%%

input: expr '\n'

{
    printf("Preorder: ");
    preorder($1);
    printf("\n");
};

expr:

    expr '+' expr { $$=make('+',$1,$3); }

| expr '*' expr { $$=make('*', $1,$3); }

| ID      { $$=make($1,NULL,NULL); }

;

%%

```

```

int main()

{

    printf("Enter expression: ");

    yyparse();

    return 0;

}

```

OUTPUT

```

C:\Windows\System32\cmd.e  ×  +  ∨
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kavet\Downloads\C03-AT1\Q2>gcc Q2.tab.c -o Q2.exe

C:\Users\kavet\Downloads\C03-AT1\Q2>Q2.exe
Enter arithmetic expression: a+b*c

Abstract Syntax Tree:
+
  a
  *
    b
    c

Preorder Traversal : + a * b c
Inorder Traversal  : a + b * c
Postorder Traversal: a b c * +

C:\Users\kavet\Downloads\C03-AT1\Q2>Q2.exe
Enter arithmetic expression: b+c*d+b

Abstract Syntax Tree:
+
  +
    b
    *
      c
      d
  b

Preorder Traversal : + + b * c d b
Inorder Traversal  : b + c * d + b
Postorder Traversal: b c d * + b +
|

```

Q3. Synthesized Attributes using YACC

Implement a Syntax-Directed Definition using synthesized attributes such as \$\$, \$1, \$2, etc. Evaluate $2 * 3 + 4$ and demonstrate the bottom-up evaluation process.

Q3.1

```
%{
#include "Q3.tab.h"

}%

%%

[0-9]+ { yylval=atoi(yytext); return NUMBER; }

[ \t] ;

\n    return '\n';

.    return yytext[0];

%%
```

Q3.y

```
%{
#include<stdio.h>

int yylex();

void yyerror(char*s){printf("Syntax Error\n");}

%}

%token NUMBER

%left '+'

%left '*'

%%
```

input:

```
expr '\n'
{
    printf("Result = %d\n",$1);
};
```

expr:

```
expr '+' term
{
    $$=$1+$3;
    printf("%d + %d = %d\n",$1,$3,$$);
}
| term { $$=$1; };
```

term:

```
term '*' factor
{
    $$=$1*$3;
    printf("%d * %d = %d\n",$1,$3,$$);
}
| factor { $$=$1; };
```

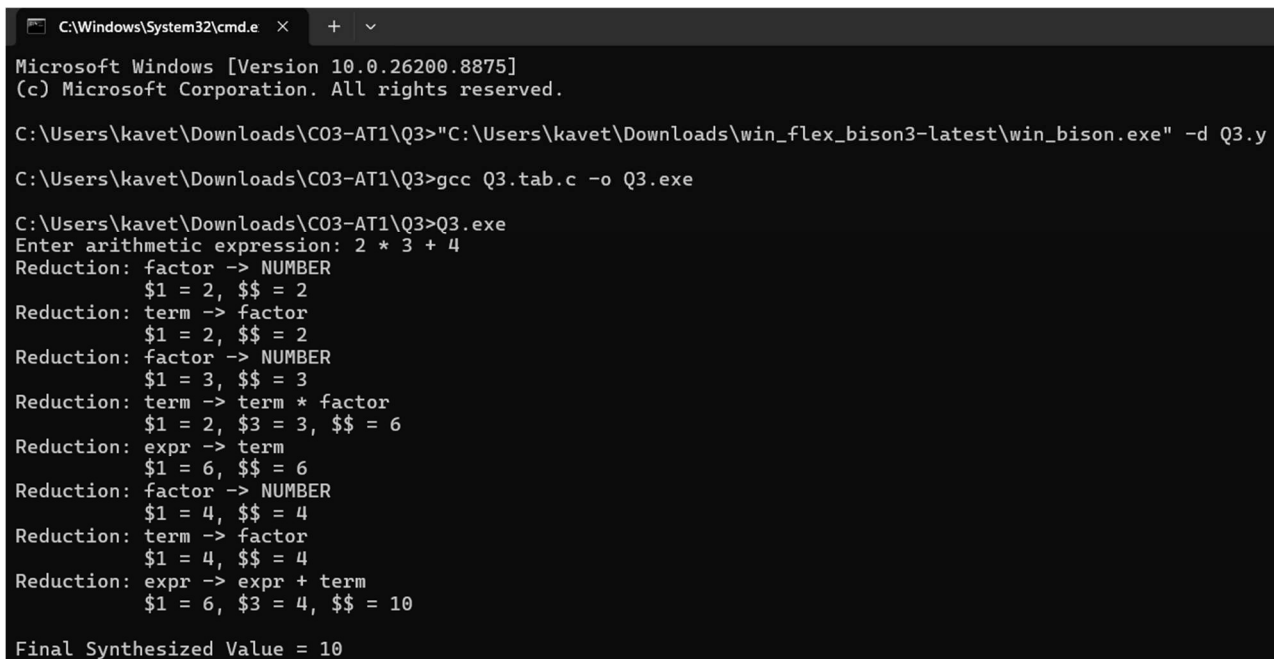
factor:

```
NUMBER { $$=$1; };
```

%%

```
int main()
{
    printf("Enter expression: ");
    yyparse();
    return 0;
}
```

OUTPUT



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kavet\Downloads\C03-AT1\Q3>"C:\Users\kavet\Downloads\win_flex_bison3-latest\win_bison.exe" -d Q3.y
C:\Users\kavet\Downloads\C03-AT1\Q3>gcc Q3.tab.c -o Q3.exe

C:\Users\kavet\Downloads\C03-AT1\Q3>Q3.exe
Enter arithmetic expression: 2 * 3 + 4
Reduction: factor -> NUMBER
           $1 = 2, $$ = 2
Reduction: term -> factor
           $1 = 2, $$ = 2
Reduction: factor -> NUMBER
           $1 = 3, $$ = 3
Reduction: term -> term * factor
           $1 = 2, $3 = 3, $$ = 6
Reduction: expr -> term
           $1 = 6, $$ = 6
Reduction: factor -> NUMBER
           $1 = 4, $$ = 4
Reduction: term -> factor
           $1 = 4, $$ = 4
Reduction: expr -> expr + term
           $1 = 6, $3 = 4, $$ = 10

Final Synthesized Value = 10
```

Q4. Type Equivalence using YACC

Design a YACC grammar to process type and variable declarations. For type A = int; type B = int; A x; B y;, determine **name equivalence** and **structural equivalence**.

Q4.1

```
%{  
  
#include "Q4.tab.h"  
  
#include<string.h>  
  
%}  
  
%%  
  
"type" return TYPE;  
  
"int" return INT;  
  
[a-zA-Z]+ {  
    yylval=strdup(yytext);  
    return ID;  
}  
  
[ \t\n] ;  
  
"=" return '=';  
  
";" return ';';  
  
%%
```

Q4.y

```
%{  
  
#include<stdio.h>  
  
#include<string.h>  
  
char A[20],B[20];
```

```

int yylex();

void yyerror(char*s){printf("Syntax Error\n");}

%}

%union { char *s; }

%token TYPE INT

%token <s> ID

%type <s> type

%%

program:

    decls

    {
        printf("\nName Equivalence: ");
        printf(strcmp(A,B)==0 ? "Equivalent\n" : "Not Equivalent\n");
        printf("Structural Equivalence: ");
        printf("Equivalent\n");
    };

decls:

    decls decl

    | decl;

decl:

    TYPE ID '=' type ';'

    {
        if(A[0]=='\0') strcpy(A,$2);
        else strcpy(B,$2);
    }

```

```

    }
| ID ID ';'

    {}

;

type:

    INT { $$=strdup("int"); };

%%

int main()
{
    A[0]='\0';
    B[0]='\0';
    printf("Enter declarations:\n");
    yyparse();
    return 0;
}

```

OUTPUT

```

C:\Users\kavet\Downloads\C03-AT1\Q4>"C:\Users\kavet\Downloads\win_flex_bison3-latest\win_bison.exe" -d Q4.y
C:\Users\kavet\Downloads\C03-AT1\Q4>Q4.exe
Enter type and variable declarations:

Example:
type A = int;
type B = int;
A x;
B y;

Enter your declarations:
type A = int;
Type declaration: A = int
type B = int;
Type declaration: B = int
A x;
Variable declaration: A x
B y;
Variable declaration: B y
^Z

All declarations processed successfully.

===== TYPE EQUIVALENCE ANALYSIS =====
Type of x = A
Type of y = B

Name Equivalence      : Not Equivalent
Structural Equivalence : Equivalent

C:\Users\kavet\Downloads\C03-AT1\Q4>|

```

Q5. Type Coercion using YACC

Design a YACC grammar for variable declarations and assignments. For float temperature; int sensor_value; temperature = sensor_value;, demonstrate the implicit **int** → **float** type conversion.

Q5.1

```
%{  
  
#include "Q5.tab.h"  
  
#include<string.h>  
  
%}  
  
%%  
  
"float" return FLOAT;  
  
"int"  return INT;  
  
[a-zA-Z_][a-zA-Z0-9_]* {  
    yylval=strdup(yytext);  
    return ID;  
}  
  
[ \t\n] ;  
  
"=" return '=';  
  
";" return ';';  
  
%%
```

Q5.y

```
%{  
  
#include<stdio.h>  
  
#include<string.h>  
  
char temp[20],sensor[20];  
  
int yylex();  
  
void yyerror(char*s){printf("Syntax Error\n");}
```

```
%}
```

```
%union { char *s; }
```

```
%token FLOAT INT
```

```
%token <s> ID
```

```
%%
```

```
program:
```

```
    FLOAT ID ';' 
```

```
    {
```

```
        strcpy(temp,$2);
```

```
    }
```

```
    INT ID ';' 
```

```
    {
```

```
        strcpy(sensor,$2);
```

```
    }
```

```
    ID '=' ID ';' 
```

```
    {
```

```
        printf("LHS Type : float\n");
```

```
        printf("RHS Type : int\n");
```

```
        printf("Conversion: int -> float\n");
```

```
        printf("Final Type: float\n");
```

```
    };
```

```
%%
```

```
int main()
```

```
{
```

```
    printf("Enter declarations and assignment:\n");
```

```
    yyparse();
```

```
    return 0;
```

```
}
```

OUTPUT

```
C:\Users\kavet\Downloads\C03-AT1\Q5>Q5.exe
Enter program fragment:

Example:
float temperature;
int sensor_value;
temperature = sensor_value;

float temperature;
Declared: float temperature
int sensor_value;
Declared: int sensor_value
temperature = sensor_value;

===== ASSIGNMENT ANALYSIS =====
Assignment: temperature = sensor_value
Left-hand type  : float
Right-hand type : int

Implicit type conversion:
int -> float
The integer value of 'sensor_value' is promoted to float.
Final type of 'temperature' = float
```

```
|
```